

PRD — "Pip" Avatar Animasi Voice Mode (iOS)

Versi: 1.0 · **Tanggal:** 12 Juli 2026 · **Status:** Draft · **Target rilis:** MVP (dasar) + v0.2 (polish) ·
Induk: PRD-Voice-Assistant (global)

1. Ringkasan

"Pip" adalah ikon/avatar animasi yang menjadi wajah asisten di voice mode. Pip harus terasa **hidup**: saat AI berbicara, animasinya bergerak **mengikuti ritme dan volume suara secara real-time** (bukan loop animasi generik), dan sepanjang waktu ia **berkedip secara natural seperti manusia**. Tujuannya: percakapan suara terasa seperti ngobrol dengan sosok, bukan menatap layar mati.

2. Masalah yang Diselesaikan

Tanpa umpan balik visual, voice mode terasa seperti bicara ke tembok: user tidak yakin AI-nya sedang mendengarkan, berpikir, atau bicara — apalagi saat ada jeda. Animasi generik (loop yang sama terus) cepat terasa palsu. Animasi yang tersinkron dengan suara sungguhan menjawab dua-duanya.

3. Goals

1. Animasi bicara tersinkron dengan audio nyata: bergerak besar saat suara keras, kecil saat pelan, diam saat senyap — delay visual < 50ms dari suara.
2. Kedipan natural: interval acak seperti manusia, bukan metronomik.
3. Setiap state percakapan punya bahasa visual yang jelas (idle / mendengarkan / berpikir / bicara / terpotong).
4. Ringan: 60fps stabil, tidak mengganggu pipeline audio.

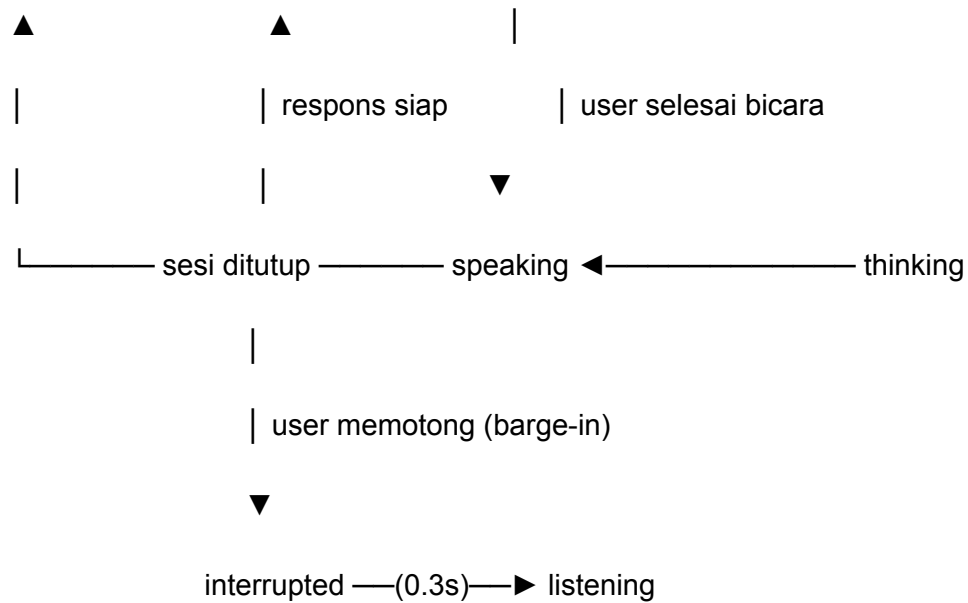
Non-Goals: lip-sync fonem sungguhan (bentuk mulut per huruf — kompleksitasnya tidak sepadan untuk ikon abstrak); avatar 3D; kustomisasi wajah oleh user (nanti saja).

4. Desain Perilaku

4.1 State Machine Pip

tap tombol voice

idle —————▶ greeting/speaking —selesai—▶ listening



4.2 Visual per state

State	Perilaku visual	Sumber sinyal
idle	"Bernafas": scale pelan 1.00↔1.03, siklus ~4 dtk + kedip	Timer
listening	Glow/ring lembut; bereaksi halus terhadap suara user (membesar sedikit saat user bicara — tanda "aku dengar kamu") + kedip	Amplitudo mic
thinking	Denyut ritmis kecil / partikel berputar pelan — pengganti ikon "🤔" di voice mode + kedip	Timer
speaking	Inti fitur: bentuk Pip (blob/mulut/waveform) bergerak mengikuti amplitudo audio TTS real-time; hentakan suku kata terlihat sebagai hentakan visual; jeda	Amplitudo playback

State	Perilaku visual	Sumber sinyal
	antar kalimat = Pip mengecil tenang + kedip tetap jalan	
interrupted	Berhenti seketika ($\leq 100\text{ms}$) → animasi "menciut" singkat → masuk listening. Sinkron dengan event interrupted dari backend	Event WS

4.3 Sinkronisasi ritme suara (speaking)

Prinsip: jangan tebak-tebakan — ukur langsung audio yang sedang diputar.

1. Chunk PCM dari WS masuk ke buffer playback (**AVAudioEngine**).
2. Pasang **tap** pada node playback (atau hitung dari chunk sebelum di-enqueue): ambil **RMS amplitude** per frame (~setiap 20–30ms).
3. Haluskan dengan **envelope follower**: attack cepat (~50ms, biar hentakan suku kata langsung terlihat), release lebih lambat (~150–200ms, biar tidak flicker).
4. Normalisasi ke 0–1, lalu petakan ke parameter visual: **scale** = $1.0 + 0.25 \times \text{level}$, intensitas glow, dan amplitudo bentuk blob/bar.
5. Opsional (v0.2 polish): pecah jadi 2–3 band frekuensi sederhana (rendah = badan membesar, tinggi = tepi bergetar) agar gerak terasa lebih "berbicara", tetap tanpa FFT berat — cukup filter sederhana.

Hasilnya: saat AI mengucapkan kalimat bersemangat, Pip ikut bergerak hidup; saat mengucapkan pelan, gerakannya lembut; saat jeda napas antar kalimat, Pip ikut "menarik napas". Ini yang membuatnya terasa mengikuti ritme, bukan loop.

4.4 Kedipan natural

Manusia berkedip rata-rata tiap 3–5 detik dengan variasi besar. Aturan Pip:

Parameter	Nilai
Interval antar kedip	Acak 2,5–6 dtk (distribusi merata, di-roll ulang tiap kedip)
Durasi satu kedip	120–180ms (tutup cepat ~40%, buka ~60% dari durasi)

Parameter	Nilai
Kedip ganda	Peluang ~15% kedip kedua menyusul dalam 300ms
Saat bicara	Kedip tetap jalan tapi ditahan saat amplitudo sedang tinggi (manusia jarang berkedip pas menekankan kata) — tunda sampai amplitudo turun
Saat interrupted	Satu kedip cepat saat transisi (kesan "eh, iya?")

Kedipan berjalan di semua state — inilah detail terkecil yang paling membuat Pip terasa bernyawa bahkan saat diam.

5. Desain Teknis (iOS)

5.1 Arsitektur

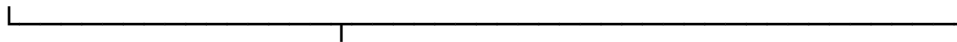
AVAudioEngine playback node

| installTap (buffer 1024 frames)



AudioLevelMeter (RMS → envelope follower) BlinkScheduler (timer acak)

| @Published level: Float (0–1) | @Published eyeOpen: CGFloat



PipViewModel (state machine + gabung sinyal)



PipView — SwiftUI TimelineView(.animation) + Canvas

(gambar blob/mata/glow per frame dari level + eyeOpen + state)

Pilihan render: **SwiftUI TimelineView + Canvas** untuk bentuk prosedural (blob/waveform yang digambar dari parameter) — paling fleksibel untuk animasi yang di-drive data. Alternatif bila desain Pip berbentuk karakter ilustrasi: **Lottie** untuk kedip/transisi state + layer Canvas terpisah untuk elemen yang audio-reactive.

5.2 Detail penting

1. **Thread:** tap audio berjalan di audio thread — hanya hitung RMS di sana, publish ke main thread lewat buffer atomik; jangan sentuh UI dari audio thread.
2. **Anti-flicker:** semua parameter visual melewati envelope follower; tidak ada nilai mentah yang langsung ke layar.
3. **Sinkron interupsi:** event **interrupted** dari WS + penghentian playback lokal harus mematikan animasi speaking dalam $\leq 100\text{ms}$ — kalau suara sudah berhenti tapi Pip masih "ngomong", ilusi hidupnya pecah seketika.
4. **Hemat baterai:** saat idle, turunkan refresh ke $\sim 20\text{fps}$ (TimelineView schedule lebih jarang); 60fps hanya saat speaking/listening.
5. **Aksesibilitas:** hormati *Reduce Motion* — ganti gerakan blob dengan perubahan opacity/glow sederhana, kedip tetap.
6. **Transisi state:** semua perpindahan pakai spring animation pendek ($0,25\text{--}0,4\text{ dtk}$) agar tidak ada lompatan visual.

5.3 Kontrak data dari backend (sudah tersedia, tanpa perubahan BE)

Semua sinyal yang dibutuhkan Pip sudah ada di desain WS voice MVP:

greeting_start/greeting_end (masuk state speaking), **listening**, **interrupted**, chunk audio binary (sumber amplitudo), dan **transcript**. **Fitur ini murni frontend** — tidak menambah beban backend sama sekali.

6. Rencana Kerja

Tahap	Pekerjaan	Estimasi	Target
1	AudioLevelMeter: tap + RMS + envelope follower, uji dengan file audio dulu	1 hari	MVP
2	PipView dasar: blob Canvas digerakkan level + state machine 5 state	1–2 hari	MVP

Tahap	Pekerjaan	Estimasi	Target
3	BlinkScheduler + integrasi kedip ke semua state	0,5 hari	MVP
4	Integrasi penuh dengan WS voice (greeting, interrupt $\leq 100\text{ms}$) + tuning attack/release	1 hari	MVP
5	Polish: multi-band sederhana, reaksi terhadap suara user saat listening, Reduce Motion, hemat baterai	1–2 hari	v0.2

Tahap 1–4 (± 4 hari) masuk **Minggu 3 MVP** (slot "greeting, transkrip, stabilkan") — Pip dasar yang sudah ritme-sync + kedip. Tahap 5 menyusul di v0.2.

7. Acceptance Criteria

1. Saat AI bicara, gerakan Pip terlihat mengikuti naik-turun suara — diuji dengan kalimat berisi penekanan ("INI penting banget!") vs kalimat datar: gerakan harus jelas berbeda.
2. Saat AI diam di tengah jeda kalimat, Pip ikut tenang (tidak terus bergerak-gerak).
3. Kedipan tidak pernah terasa berpola — diamati 1 menit, interval terlihat acak, sesekali kedip ganda.
4. Barge-in: dari user mulai bicara sampai Pip berhenti "ngomong" $\leq 100\text{ms}$ setelah audio berhenti.
5. 60fps stabil di voice mode (cek Instruments), tanpa audio glitch akibat tap.
6. Dengan Reduce Motion aktif, semua state tetap terbedakan tanpa gerakan besar.

8. Risiko & Mitigasi

Risiko	Mitigasi
Tap audio mengganggu playback (glitch/latensi)	Buffer tap kecil (1024), hanya RMS di audio thread; bila masih bermasalah, hitung RMS dari chunk PCM sebelum enqueue (tanpa tap sama sekali)

Risiko	Mitigasi
Animasi terasa "telat" dari suara	Ukur delay end-to-end; bila perlu hitung level dari chunk pre-enqueue dan jadwalkan sesuai posisi playback
Blob prosedural susah dibuat "lucu/berkarakter"	Prototipe cepat 2–3 gaya (blob, ring waveform, wajah minimal) → pilih satu sebelum tahap 4
Baterai boros saat sesi panjang	Frame rate adaptif per state; profil dengan Instruments di sesi 10 menit